ITA0605- Machine Learning
List of Lab Exercises

10.Write a Python Program to Implement Expectation & Maximization Algorithm

Aim

To implement the Expectation-Maximization (EM) Algorithm for clustering data using a Gaussian Mixture Model (GMM) in Python.

Algorithm

1. Import the required libraries.
2. Load a dataset (Iris dataset).
3. Select numerical features for clustering.
4. Standardize the dataset.
5. Initialize the Gaussian Mixture Model with the desired number of clusters.
6. Train the model using the Expectation-Maximization (EM) algorithm.
7. Predict the cluster labels.
8. Display the cluster assignments.
9. Visualize the clustered data.

Program Code

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler from sklearn.mixture

import GaussianMixture

iris = load_iris()
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)

scaler = StandardScaler()

X = scaler.fit_transform(df)
gmm = GaussianMixture(

    n_components=3,

    random_state=42

)

gmm.fit(X)
clusters = gmm.predict(X)

df["Cluster"] = clusters


print("First 10 Records with Cluster Labels:\n") print(df.head(10))

plt.figure(figsize=(8,6))

plt.scatter(

    X[:,0],

    X[:,1],

    c=clusters, cmap='viridis',



s=60

)


plt.title("Expectation Maximization Clustering") plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.colorbar(label="Cluster") plt.show()
```

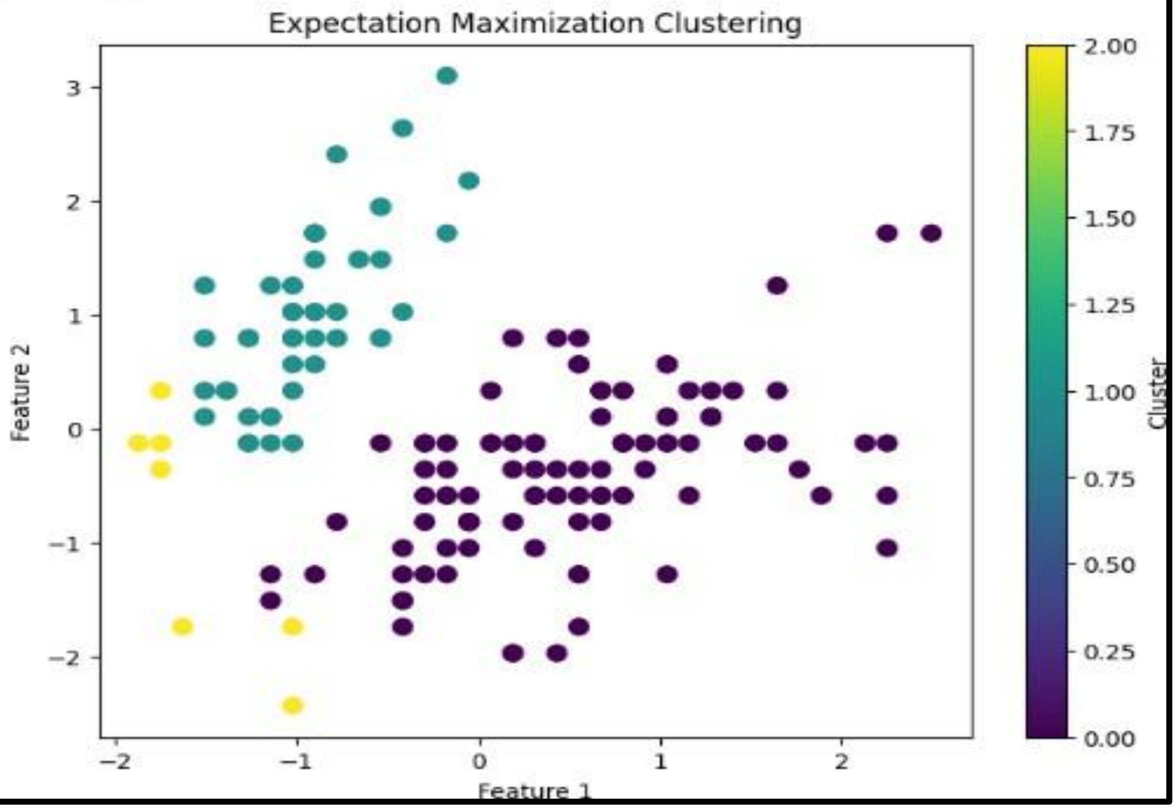
Output:

```python
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture

# Load Iris Dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Standardize the data
scaler = StandardScaler()
X = scaler.fit_transform(df)

# Apply Gaussian Mixture Model (Expectation Maximization)
gmm = GaussianMixture(
    n_components=3,
    random_state=42
)

gmm.fit(X)

# Predict Cluster Labels
clusters = gmm.predict(X)

# Add Cluster Labels to DataFrame
df["Cluster"] = clusters

# Display first 10 records
print("First 10 Records with Cluster Labels:\n")
print(df.head(10))
```

```python
print(df.head(10))

# Plot Clusters
plt.figure(figsize=(8,6))

plt.scatter(
    X[:,0],
    X[:,1],
    c=clusters,
    cmap='viridis',
    s=60
)

plt.title("Expectation Maximization Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.colorbar(label="Cluster")
plt.show()
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2
5                5.4               3.9                1.7               0.4
6                4.6               3.4                1.4               0.3
7                5.0               3.4                1.5               0.2
8                4.4               2.9                1.4               0.2
9                4.9               3.1                1.5               0.1

   Cluster
0        1
1        1
```

```
   Cluster
0        1
1        1
2        1
3        1
4        1
5        1
6        1
7        1
8        2
9        1
```



Dataset 2:

Program Code:

Model

import numpy as np

from sklearn.mixture import

GaussianMixture

X = np.array([

[2, 20],

```
    [3, 25],
    [4, 30],
    [5, 40],
    [6, 50],
    [8, 65],
    [10, 75],
    [12, 82],
    [14, 90],
    [16, 96]
])
model =
GaussianMixture(n_components=2,
random_state=42)
model.fit(X)
labels = model.predict(X)
new_customer = np.array([[9, 70]])
new_cluster =
model.predict(new_customer)
print("DATASET 2: CUSTOMER
SEGMENTATION")
print("--------------------------------")
```

```
for i in range(len(X)):
    print("Income:", X[i][0],
          "Lakhs, Spending Score:", X[i][1],
          "-> Cluster:", labels[i])


print("\nNew Customer:")
print("Income = ₹9 Lakhs")
print("Spending Score = 70")
print("Predicted Cluster =", new_cluster[0])
```

Output:

```
print("Predicted Cluster =", new_cluster[0])

... DATASET 2: CUSTOMER SEGMENTATION
    ---------------------------------
    Income: 2 Lakhs, Spending Score: 20 -> Cluster: 0
    Income: 3 Lakhs, Spending Score: 25 -> Cluster: 0
    Income: 4 Lakhs, Spending Score: 30 -> Cluster: 0
    Income: 5 Lakhs, Spending Score: 40 -> Cluster: 0
    Income: 6 Lakhs, Spending Score: 50 -> Cluster: 0
    Income: 8 Lakhs, Spending Score: 65 -> Cluster: 0
    Income: 10 Lakhs, Spending Score: 75 -> Cluster: 1
    Income: 12 Lakhs, Spending Score: 82 -> Cluster: 1
    Income: 14 Lakhs, Spending Score: 90 -> Cluster: 1
    Income: 16 Lakhs, Spending Score: 96 -> Cluster: 1

    New Customer:
    Income = ₹9 Lakhs
    Spending Score = 70
    Predicted Cluster = 0
```

Dataset 3:

Program Code:

Model

```
import numpy as np

from sklearn.mixture import

GaussianMixture

X = np.array([

    [800, 30],

    [900, 35],

    [1000, 40],

    [1200, 50],

    [1400, 60],

    [1600, 75],

    [1800, 90],

    [2000, 105],

    [2200, 120],

    [2400, 135]

])

model =

GaussianMixture(n_components=2,

random_state=42)
```

```
model.fit(X)

labels = model.predict(X)

new_house = np.array([[1700, 82]])

new_cluster = model.predict(new_house)

print("DATASET 3: HOUSE SIZE

CLUSTERING")

print("---------------------------------")


for i in range(len(X)):

    print("House Size:", X[i][0],

          "sq.ft, Price:", X[i][1],

          "Lakhs -> Cluster:", labels[i])


print("\nNew House:")

print("House Size = 1700 sq.ft")

print("Price = ₹82 Lakhs")

print("Predicted Cluster =", new_cluster[0])
```

Output:

```
print("Predicted Cluster =", new_cluster[0])

DATASET 3: HOUSE SIZE CLUSTERING
---------------------------------
House Size: 800 sq.ft, Price: 30 Lakhs -> Cluster: 0
House Size: 900 sq.ft, Price: 35 Lakhs -> Cluster: 0
House Size: 1000 sq.ft, Price: 40 Lakhs -> Cluster: 0
House Size: 1200 sq.ft, Price: 50 Lakhs -> Cluster: 0
House Size: 1400 sq.ft, Price: 60 Lakhs -> Cluster: 1
House Size: 1600 sq.ft, Price: 75 Lakhs -> Cluster: 1
House Size: 1800 sq.ft, Price: 90 Lakhs -> Cluster: 1
House Size: 2000 sq.ft, Price: 105 Lakhs -> Cluster: 1
House Size: 2200 sq.ft, Price: 120 Lakhs -> Cluster: 1
House Size: 2400 sq.ft, Price: 135 Lakhs -> Cluster: 1

New House:
House Size = 1700 sq.ft
Price = ₹82 Lakhs
Predicted Cluster = 1
```

Dataset 4:

Program Code:

Model

import numpy as np

from sklearn.mixture import

GaussianMixture

X = np.array([

    [1, 3.0],

    [2, 3.8],

    [3, 4.8],

    [4, 6.0],

    [5, 7.5],

    [6, 9.0],

    [7, 10.8],

```
    [8, 12.6],
    [9, 14.5],
    [10, 16.5]
])
model =
GaussianMixture(n_components=2,
random_state=42)
model.fit(X)
labels = model.predict(X)
new_employee = np.array([[7, 11.0]])
new_cluster =
model.predict(new_employee)
print("DATASET 4: EMPLOYEE
SALARY ANALYSIS")
print("-----------------------------------")
for i in range(len(X)):
    print("Experience:", X[i][0],
          "Years, Salary: ₹", X[i][1],
          "Lakhs -> Cluster:", labels[i])

print("\nNew Employee:")
print("Experience = 7 years")
```

print("Salary = ₹11 Lakhs")

print("Predicted Cluster =", new_cluster[0])

Output:

```
print("Predicted Cluster =", new_cluster[0])

... DATASET 4: EMPLOYEE SALARY ANALYSIS
    -----------------------------------
    Experience: 1.0 Years, Salary: ₹ 3.0 Lakhs -> Cluster: 0
    Experience: 2.0 Years, Salary: ₹ 3.8 Lakhs -> Cluster: 0
    Experience: 3.0 Years, Salary: ₹ 4.8 Lakhs -> Cluster: 0
    Experience: 4.0 Years, Salary: ₹ 6.0 Lakhs -> Cluster: 0
    Experience: 5.0 Years, Salary: ₹ 7.5 Lakhs -> Cluster: 0
    Experience: 6.0 Years, Salary: ₹ 9.0 Lakhs -> Cluster: 1
    Experience: 7.0 Years, Salary: ₹ 10.8 Lakhs -> Cluster: 1
    Experience: 8.0 Years, Salary: ₹ 12.6 Lakhs -> Cluster: 1
    Experience: 9.0 Years, Salary: ₹ 14.5 Lakhs -> Cluster: 1
    Experience: 10.0 Years, Salary: ₹ 16.5 Lakhs -> Cluster: 1

    New Employee:
    Experience = 7 years
    Salary = ₹11 Lakhs
    Predicted Cluster = 1
```

Dataset 5:

Program Code:

Model

import numpy as np

from sklearn.mixture import

GaussianMixture

X = np.array([

[40, 4.5],

[50, 5.0],

[60, 5.6],

[70, 6.5],

[80, 7.8],

```
    [90, 9.2],

    [100, 10.8],

    [110, 12.5],

    [120, 14.3],

    [130, 16.2]

])

model =

GaussianMixture(n_components=2,

random_state=42)

model.fit(X)

labels = model.predict(X)

new_vehicle = np.array([[95, 10.0]])

new_cluster = model.predict(new_vehicle)

print("DATASET 5: VEHICLE SPEED

ANALYSIS")

print("----------------------------------")



for i in range(len(X)):

    print("Speed:", X[i][0],

          "km/h, Fuel Consumption:", X[i][1],

          "L/100 km -> Cluster:", labels[i])
```

print("\nNew Vehicle:")

print("Speed = 95 km/h")

print("Fuel Consumption = 10.0 L/100 km")

print("Predicted Cluster =", new_cluster[0])

Output:

```
print("Predicted Cluster =", new_cluster[0])

... DATASET 5: VEHICLE SPEED ANALYSIS
----------------------------------
Speed: 40.0 km/h, Fuel Consumption: 4.5 L/100 km -> Cluster: 0
Speed: 50.0 km/h, Fuel Consumption: 5.0 L/100 km -> Cluster: 0
Speed: 60.0 km/h, Fuel Consumption: 5.6 L/100 km -> Cluster: 0
Speed: 70.0 km/h, Fuel Consumption: 6.5 L/100 km -> Cluster: 0
Speed: 80.0 km/h, Fuel Consumption: 7.8 L/100 km -> Cluster: 0
Speed: 90.0 km/h, Fuel Consumption: 9.2 L/100 km -> Cluster: 0
Speed: 100.0 km/h, Fuel Consumption: 10.8 L/100 km -> Cluster: 1
Speed: 110.0 km/h, Fuel Consumption: 12.5 L/100 km -> Cluster: 1
Speed: 120.0 km/h, Fuel Consumption: 14.3 L/100 km -> Cluster: 1
Speed: 130.0 km/h, Fuel Consumption: 16.2 L/100 km -> Cluster: 1

New Vehicle:
Speed = 95 km/h
Fuel Consumption = 10.0 L/100 km
Predicted Cluster = 0
```

Result

The Expectation-Maximization (EM) Algorithm was successfully implemented using the Gaussian Mixture Model (GMM). The model clustered the Iris dataset into three groups, and the cluster assignments were displayed and visualized successfully